

Post-Quantum and Multi-Password PAKEs in IEEE 802.11

Mathy Vanhoef

Workshop on Password Authenticated Key Exchange and Password Security & Usability (PAKE'26). 2 October 2026, Darmstadt, Germany.

Agenda

- › Post-Quantum PAKEs in IEEE 802.11: issues encountered
- › Towards Multi-Password PAKEs in IEEE 802.11

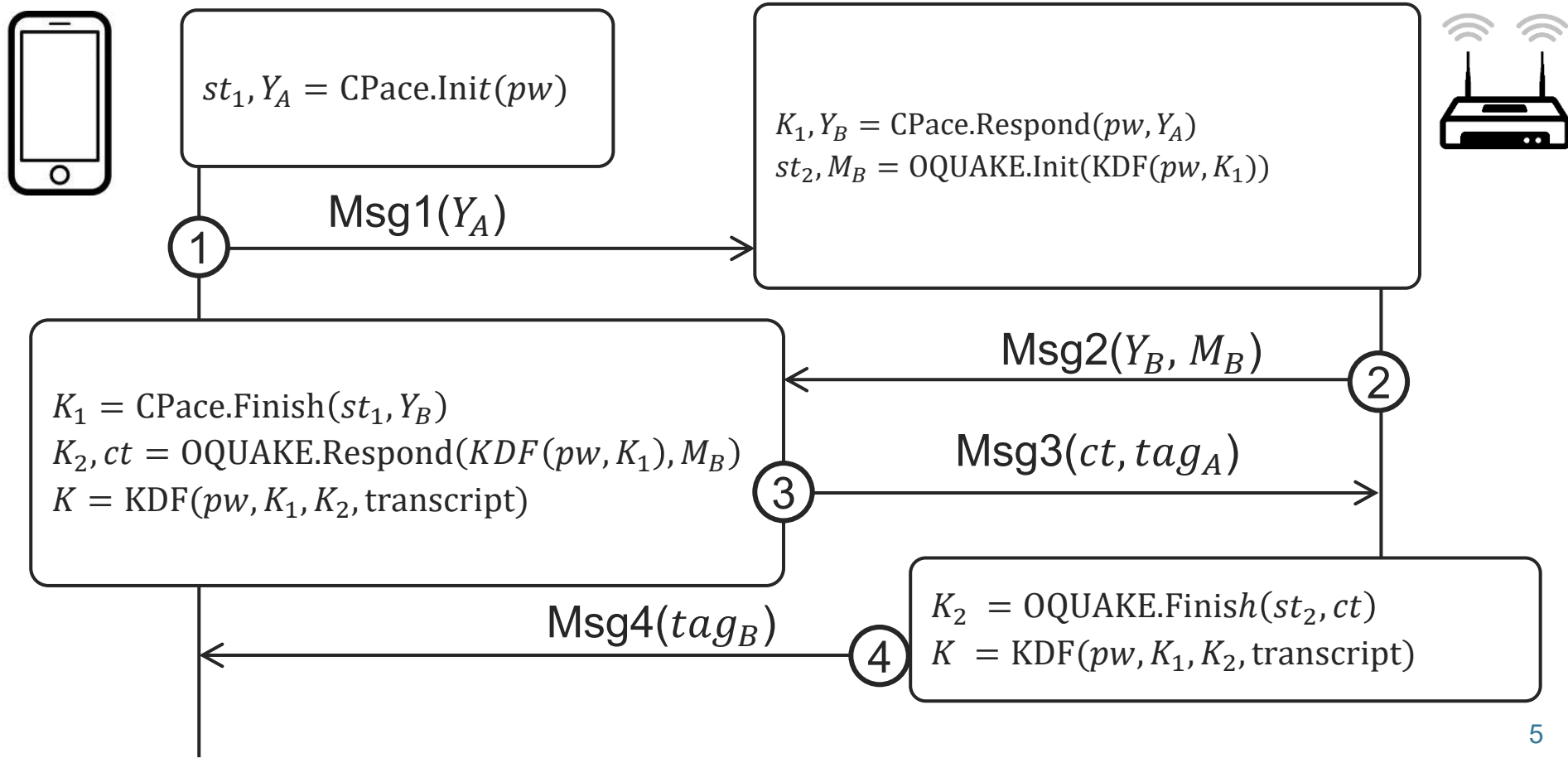


Post-Quantum PAKEs in IEEE 802.11

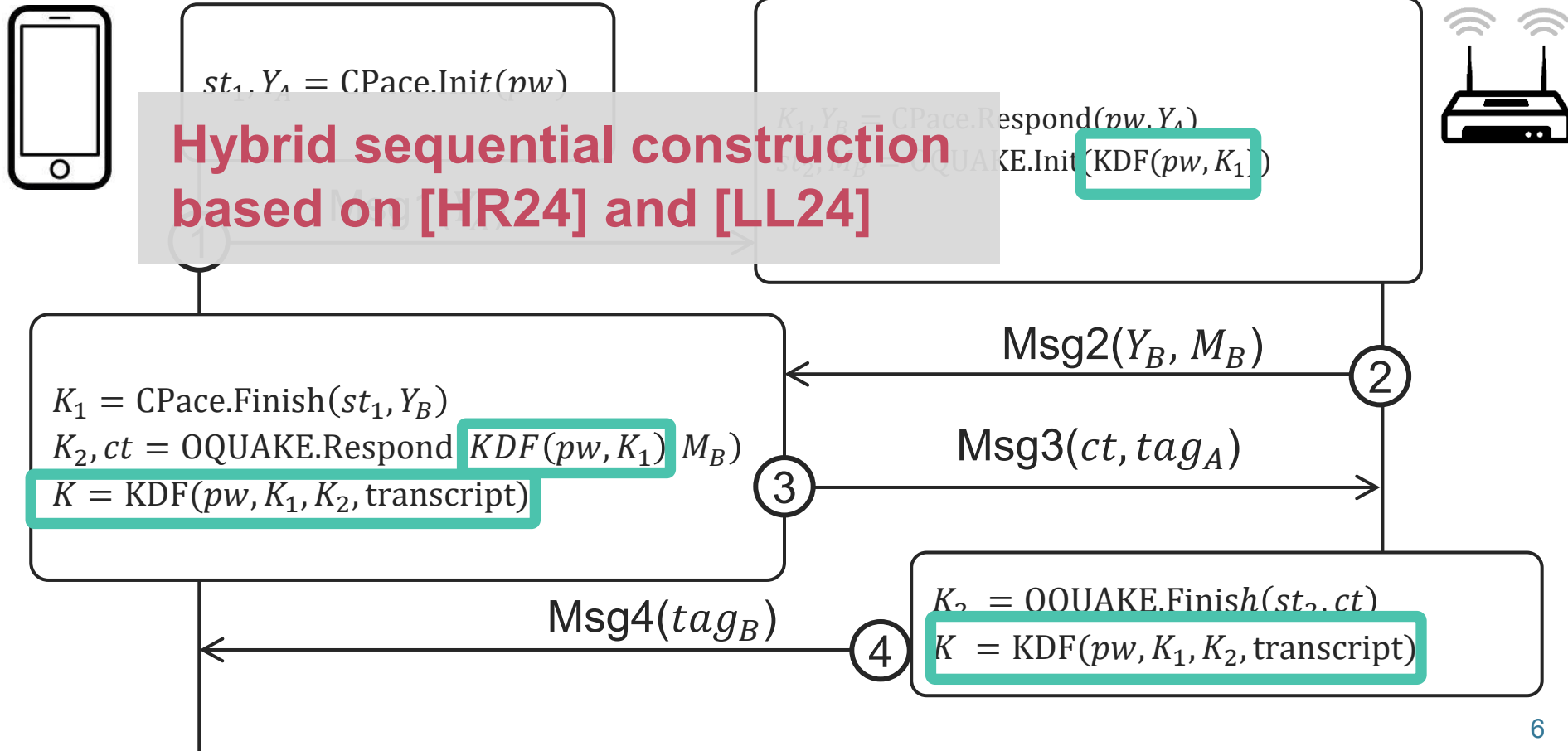
IEEE 802.11 is looking for a PQC-PAKE!

- › The IEEE 802.11 wants to have a PQC-PAKE [\[TGbt\]](#)
- › As we saw [this morning](#), there are a lot of PQC-PAKEs
- › Which one should 802.11 pick?
 - › Ideally the winner of a competition. But there is none...
 - › Alternatively, a standardized one. But there is none...
 - › Alternatively, one from another widely-used protocol? There's none...
 - › Alternatively, one that is currently *being* standardized...
- › That only leaves one option: [CPaceOQUAKE](#)
 - › So far, all discussions in 802.11 involved this protocol

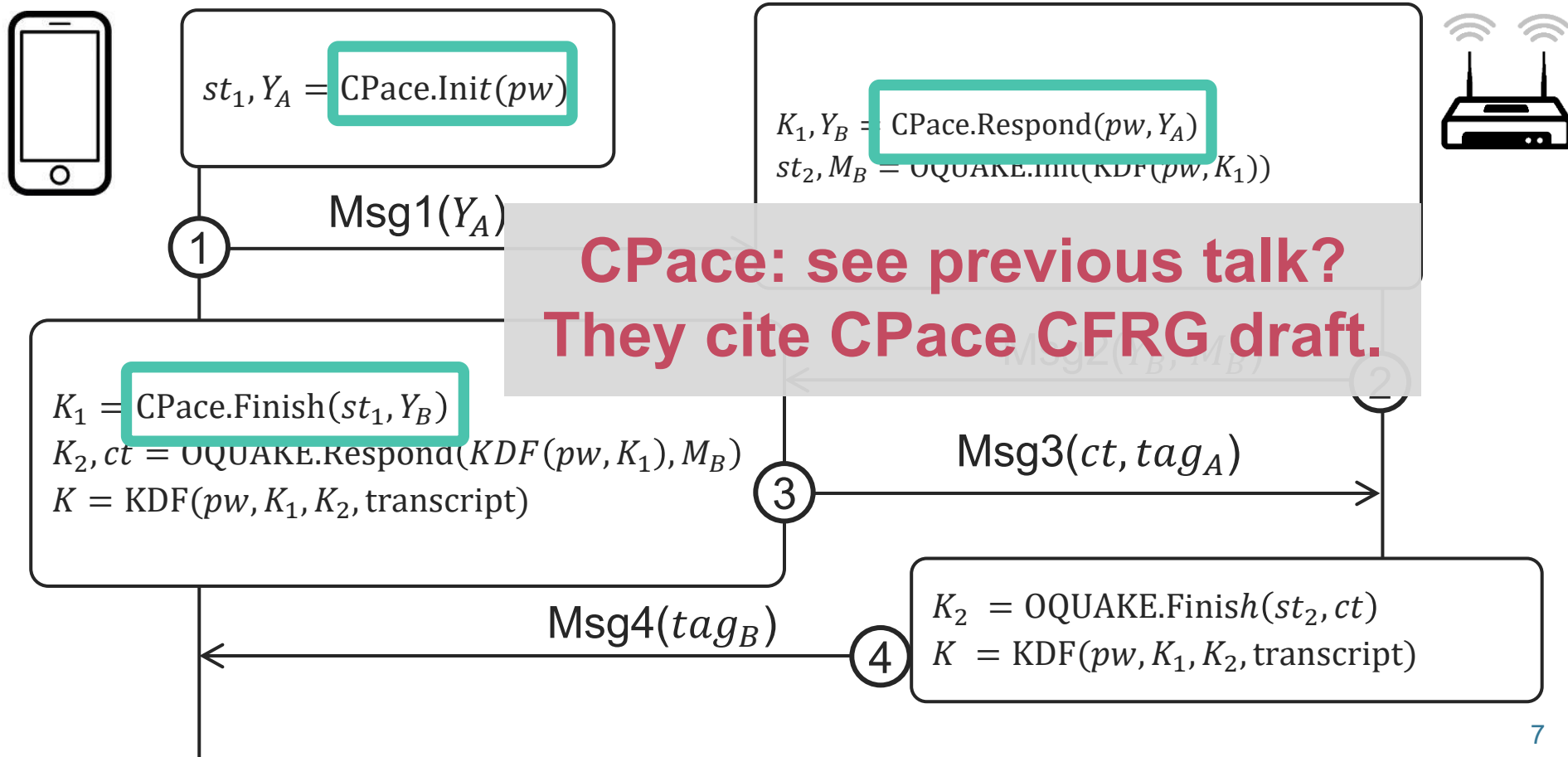
CPaceOQUAKE (4-message variant)



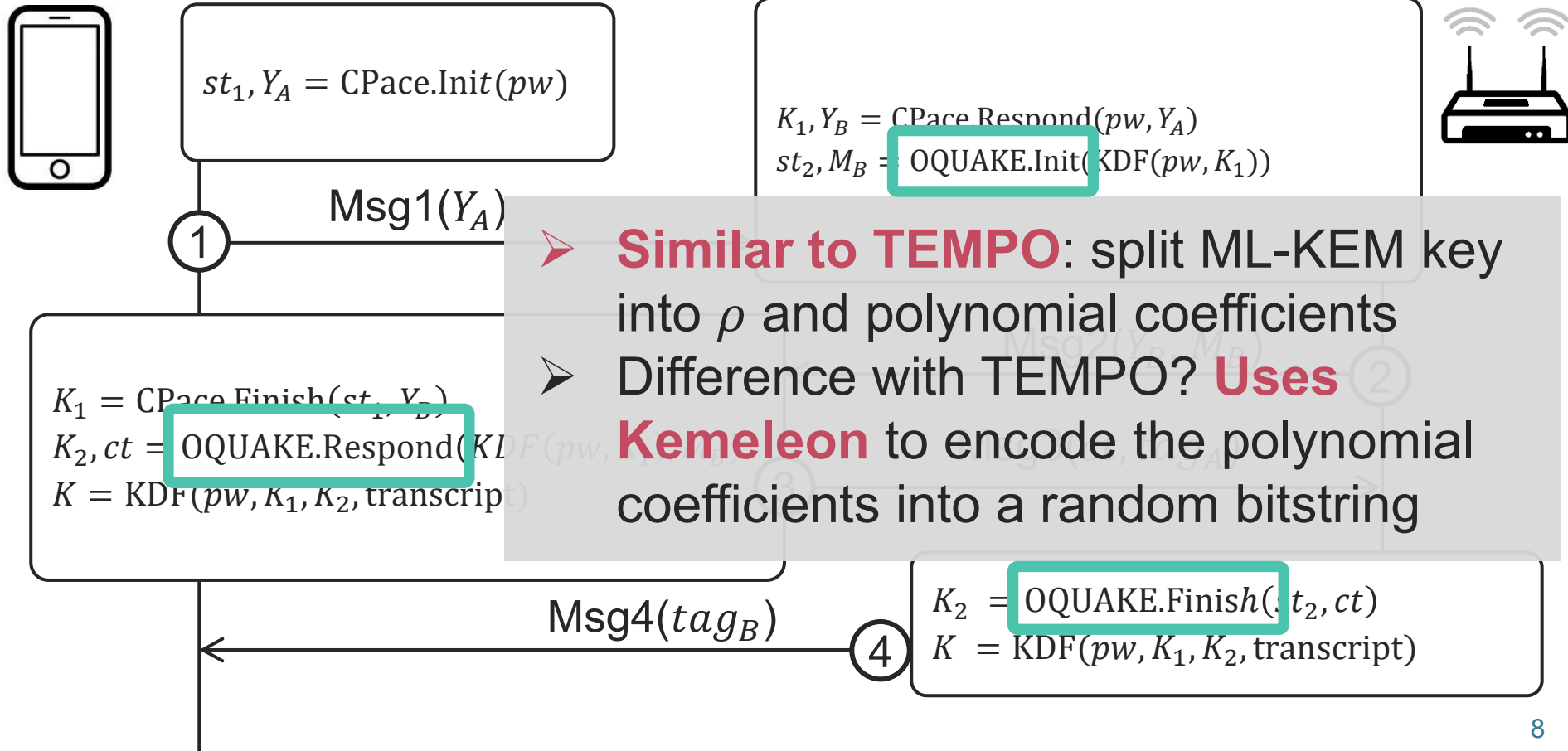
CPaceOQUAKE (4-message variant)



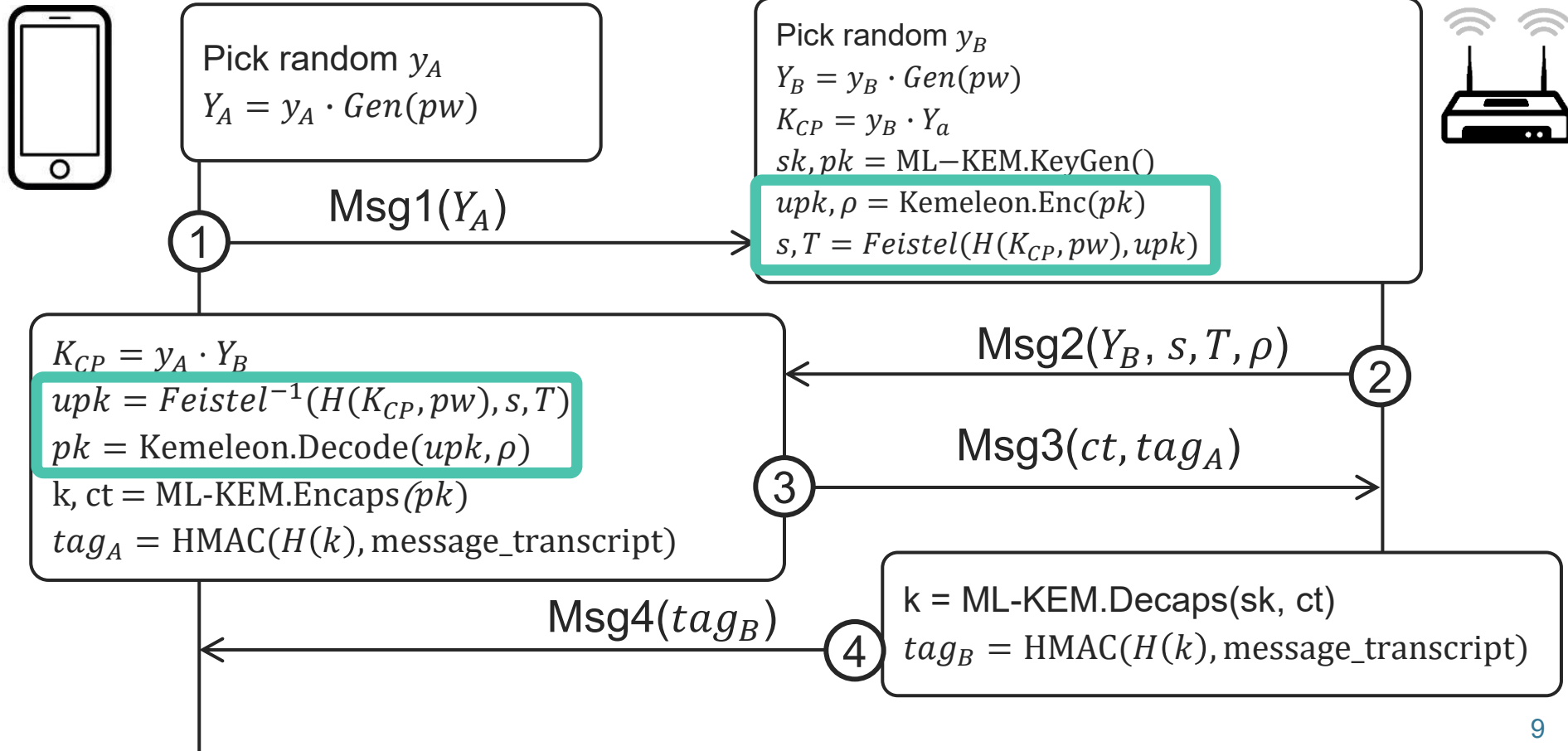
CPaceOQUAKE (4-message variant)



CPaceOQUAKE (4-message variant)



CPaceOQUAKE (4-message variant)



Background: ML-KEM public keys

Public ML-KEM key is a set of k polynomials:

- › Each polynomial has $n = 256$ coefficients
- › Each coefficient $\in Z_q$
- › Where $q = 3329$

CPaceOQUAKE uses Kemeleon Encoding

High-level idea:

- › Encode all the coefficients into a single number r
 - › Add random value to “fill” the space between the maximum value of the number r and the next power of two
 - › Send the resulting number
- Not perfect random uniform, but within statistical distance of 2^{-t} from uniform, where t is a security parameter

Kameleon encoding

Input: poly coefficients a
Output: bitstring number r

$$r = 0$$

$$b = 2996 \quad \# \text{ ceil}(n \cdot \log_2(q))$$

$$t = 76 \quad \# \text{ stat. distance from uniform and byte-aligned}$$

for i from 1 to n :

$$r = r + q^{i-1} \cdot a_i$$

$$m = \text{rand} \left(0, \left\lfloor \frac{2^{b+t} - 1 - r}{q^n} \right\rfloor \right)$$

$$\text{return } r + m \cdot q^n$$

Each a_i is secret: use constant-time multiplication.

Kameleon encoding

Input: poly coefficients a
Output: bitstring number r

$$r = 0$$

$$b = 2996 \quad \# \text{ ceil}(n \cdot \log_2(q))$$

$$t = 76 \quad \# \text{ stat. distance from uniform and byte-aligned}$$

for i from 1 to n :

$$r = r + q^{i-1} \cdot a_i$$

$$m = \text{rand} \left(0, \left[\frac{2^{b+t} - 1 - r}{q^n} \right] \right)$$

$$\text{return } r + m \cdot q^n$$

**r is secret & large denominator
makes division expensive...**

Kemeleon encoding: tweaked

Input: poly coefficients a
Output: bitstring number r

$$r = 0, t = 76, b = 2996$$

$$k = \left\lfloor 2^{b+t} / q^n \right\rfloor$$
$$s = 2^{b+t} \bmod q^n$$

for i from 1 to n :

$$r = r + q^{i-1} \cdot a_i$$

if $r < s$: $M = k$
else: $M = k - 1$

$$m = \text{rand}(0, M)$$

$$\text{return } r + m \cdot q^n$$

**This is faster & now easier to
do in constant time 😊**

Kemeleon encoding: tweaked

Input: poly coefficients a
Output: bitstring number r

$$r = 0, t = 76, b = 2996$$

$$k = \left\lfloor 2^{b+t} / q^n \right\rfloor$$

$$s = 2^{b+t} \bmod q^n$$

for i from 1 to n :

$$r = r + q^{i-1} \cdot a_i$$

if $r < s$: $M = k$

else: $M = k - 1$

$$m = \text{rand}(0, M)$$

$$\text{return } r + m \cdot q^n$$

Technically this leaks: rand() may use rejection sampling with M secret...

Kameleon encoding: tweaked

Input: poly coefficients a
Output: bitstring number r

$$r = 0, t = 76, b = 2996$$

$$k = \left\lfloor 2^{b+t} / q^n \right\rfloor$$

$$s = 2^{b+t} \bmod q^n$$

for i from 1 to n :

$$r = r + q^{i-1} \cdot a_i$$

if $r < s$: $m = \text{rand}(0, k)$

else: $m = \text{rand}(0, k - 1)$

return $r + m \cdot q^n$

Kemeleon encoding: final

Input: poly coefficients a
Output: bitstring number r

$$r = 0, t = 76, b = 2996$$

$$k = \left\lfloor 2^{b+t} / q^n \right\rfloor$$

$$s = 2^{b+t} \bmod q^n$$

for i from 1 to n :

$$r = r + q^{i-1} \cdot a_i$$

$$m_1 = \text{rand}(0, k)$$

$$m_2 = \text{rand}(0, k - 1)$$

$$m = \text{constant-time-select}(r < s, m_1, m_2)$$

$$\text{return } r + m \cdot q^n$$

Kemeleon decoding

$$r = r \% q^n$$

for i from 1 to n :

$$a_i = r \% q$$

$$r = \lfloor r/q \rfloor$$

return a

Kemeleon decoding

$r = r \% q^n$ # not needed

for i from 1 to n :

$$a_i = r \% q$$

$$r = \lfloor r/q \rfloor$$

return a

Ensure constant-time division, since numerators are secret

› q is public: use reciprocal & multiplications/shifts

What about the IEEE 802.11?

For editorial reasons in the draft RFC, CPaceOQUAKE now makes the client initiate both CPace and OQUAKE

- › Done to cleanly separate CPace and OQUAKE in the draft
- › Including key confirmation, this means 5 messages...

This causes discussions in IEEE 802.11: several feel it's now essential to move to 5-message variant as well...

- › But from a security perspective, this is *not* needed!

Including CPaceOQUAKE into IEEE 802.11?

Including CPaceOQUAKE into IEEE 802.11 faces obstacles:

- › Ideally, 802.11 doesn't define its own crypto primitives
 - › But CPaceOQUAKE is not yet an RFC and/or in a stable state
- › And the above issues:
 - › Use 4 or 5 messages?
 - › How to ensure Kemeleon in constant-time in practice?
- › Want to help review latest IEEE 802.11 submissions
 - › Ask ChatGPT to extract the crypto operations only 😊 (high thinking)
 - › Or contact me 😊

Multi-Password PAKEs (for IEEE 802.11)

Introduction: Multiple WPA2 passwords

A single network name but multiple passwords

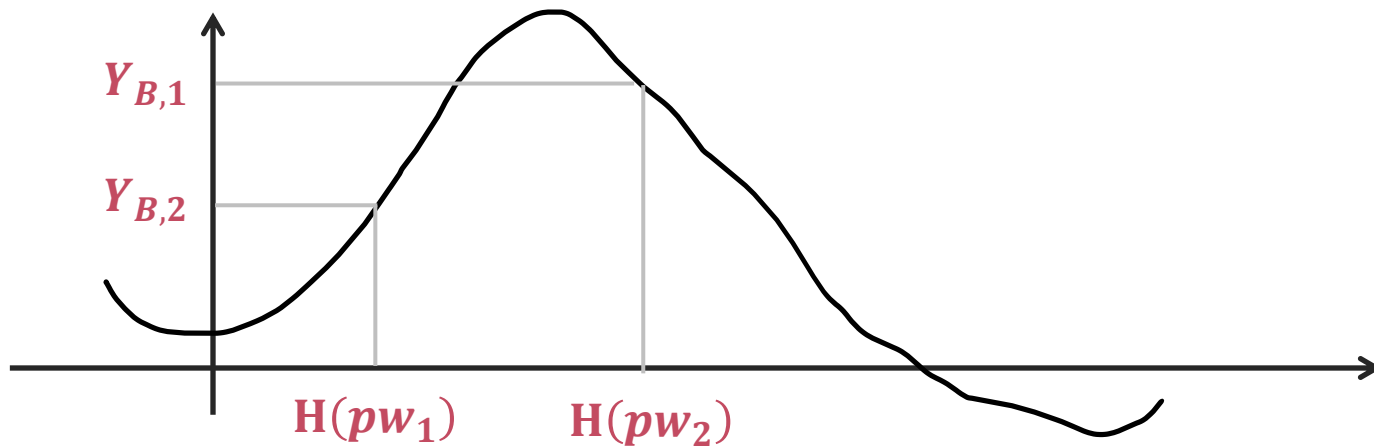
- › Use case: **guests get a different password**
 - ›› Devices connect to same network, but are put in different VLANs
- › Use case: **all users or devices get a different password**
 - ›› Infer identity from used password, can again have different VLANs
 - ›› Revoke/change individual passwords, e.g., hotels, employees,...
 - ›› **Malicious insider** can't create rogue clone of the network

Multi-password WPA2 in practice

Implemented by practically **all vendors!**

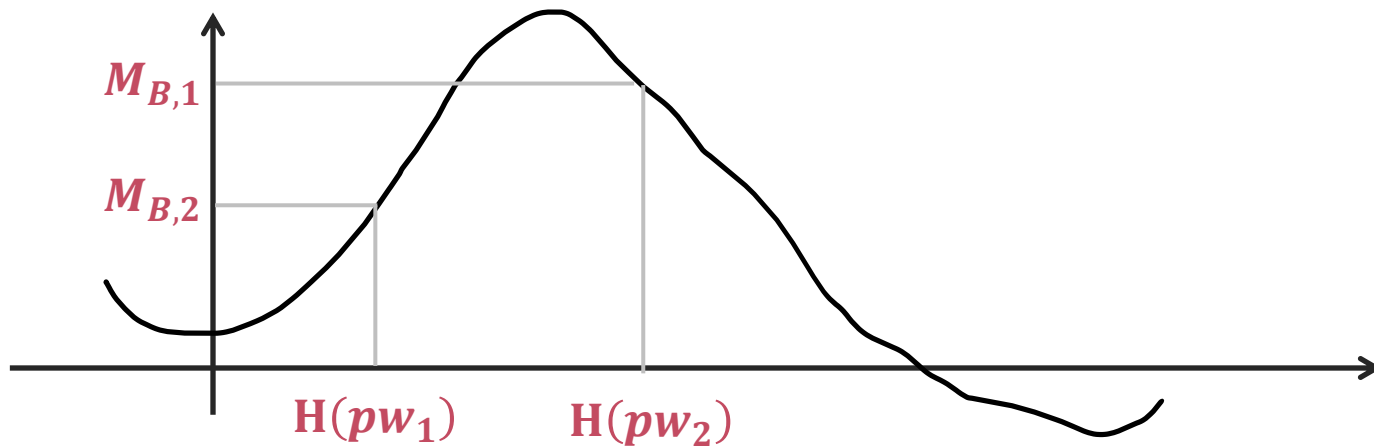
- › Possible because the AP can do an 'on-the-fly dictionary attack' against WPA2 when receiving the first client reply
- › Downside: network-side must loop through all passwords
- › **Not possible with WPA3** since its security is (now) better
- › How to do this? Post-quantum follow-up from last year:

Core idea is polynomial interpolation - CPace case



- › Encode elliptic curve points using Elligator Squared or similar.
- › **Send poly coefficients** to the client. Client recovers $Y_{B,i}$.

Core idea is polynomial interpolation - OQUAKE



- › Use **Kemeleon** to encode ML-KEM key as random bitstring
- › To prevent attacks like TEMPO, probably need to **use one rho over all ML-KEM keys**, and send rho separately

Hybrid Multi-PW PAKE



$st_1, Y_A = \text{CPace.Init}(pw)$

1

Msg1(Y_A)

For all passwords i :

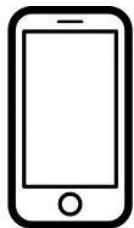
$K_{1,i}, Y_{B,i} = \text{CPace.Respond}(pw_i, Y_A)$

$\text{points}_c += (\text{H}(pw_i), Y_{B,i})$

$\text{poly}_c = \text{interpolate}(\text{points}_c)$



Hybrid Multi-PW PAKE



$st_1, Y_A = \text{CPace.Init}(pw)$

1

Msg1(Y_A)



For all passwords i :

$K_{1,i}, Y_{B,i} = \text{CPace.Respond}(pw_i, Y_A)$

$\text{points}_c += (\text{H}(pw_i), Y_{B,i})$

$st_{2,i}, M_{B,i} = \text{OQUAKE.Init}(\text{KDF}(pw_i, K_{1,i}))$

$\text{points}_q += (\text{H}(pw_i), \text{Kameleon.Enc}(M_{B,i}))$

$\text{poly}_c = \text{interpolate}(\text{points}_c)$

$\text{poly}_q = \text{interpolate}(\text{points}_q)$



Hybrid Multi-PW PAKE



$st_1, Y_A = \text{CPace.Init}(pw)$

1

Msg1(Y_A)

$Y_B = \text{poly}_c(\text{H}(pw))$

$K_1 = \text{CPace.Finish}(st_1, Y_B)$

For all passwords i :

$K_{1,i}, Y_{B,i} = \text{CPace.Respond}(pw_i, Y_A)$

$\text{points}_c += (\text{H}(pw_i), Y_{B,i})$

$st_{2,i}, M_{B,i} = \text{OQUAKE.Init}(\text{KDF}(pw_i, K_{1,i}))$

$\text{points}_q += (\text{H}(pw_i), \text{Kameleon.Enc}(M_{B,i}))$

$\text{poly}_c = \text{interpolate}(\text{points}_c)$

$\text{poly}_q = \text{interpolate}(\text{points}_q)$



Msg2($\text{poly}_c, \text{poly}_q$)

2

Hybrid Multi-PW PAKE



$st_1, Y_A = \text{CPace.Init}(pw)$

1

Msg1(Y_A)

$Y_B = \text{poly}_c(H(pw))$
 $M_B = \text{poly}_q(H(pw))$
 $K_1 = \text{CPace.Finish}(st_1, Y_B)$
 $K_2, ct = \text{OQUAKE.Respond}(KDF(pw, K_1), M_B)$

For all passwords i :

$K_{1,i}, Y_{B,i} = \text{CPace.Respond}(pw_i, Y_A)$

$\text{points}_c += (H(pw_i), Y_{B,i})$

$st_{2,i}, M_{B,i} = \text{OQUAKE.Init}(KDF(pw_i, K_{1,i}))$

$\text{points}_q += (H(pw_i), \text{Kameleon.Enc}(M_{B,i}))$

$\text{poly}_c = \text{interpolate}(\text{points}_c)$

$\text{poly}_q = \text{interpolate}(\text{points}_q)$



Msg2($\text{poly}_c, \text{poly}_q$)

2

Hybrid Multi-PW PAKE



$st_1, Y_A = \text{CPace.Init}(pw)$

1

Msg1(Y_A)

$Y_B = \text{poly}_c(H(pw))$

$M_B = \text{poly}_q(H(pw))$

$K_1 = \text{CPace.Finish}(st_1, Y_B)$

$K_2, ct = \text{OQUAKE.Respond}(KDF(pw, K_1), M_B)$

$K = KDF(pw, K_1, K_2, \text{transcript})$

For all passwords i :

$K_{1,i}, Y_{B,i} = \text{CPace.Respond}(pw_i, Y_A)$

$\text{points}_c += (H(pw_i), Y_{B,i})$

$st_{2,i}, M_{B,i} = \text{OQUAKE.Init}(KDF(pw_i, K_{1,i}))$

$\text{points}_q += (H(pw_i), \text{Kameleon.Enc}(M_{B,i}))$

$\text{poly}_c = \text{interpolate}(\text{points}_c)$

$\text{poly}_q = \text{interpolate}(\text{points}_q)$

2

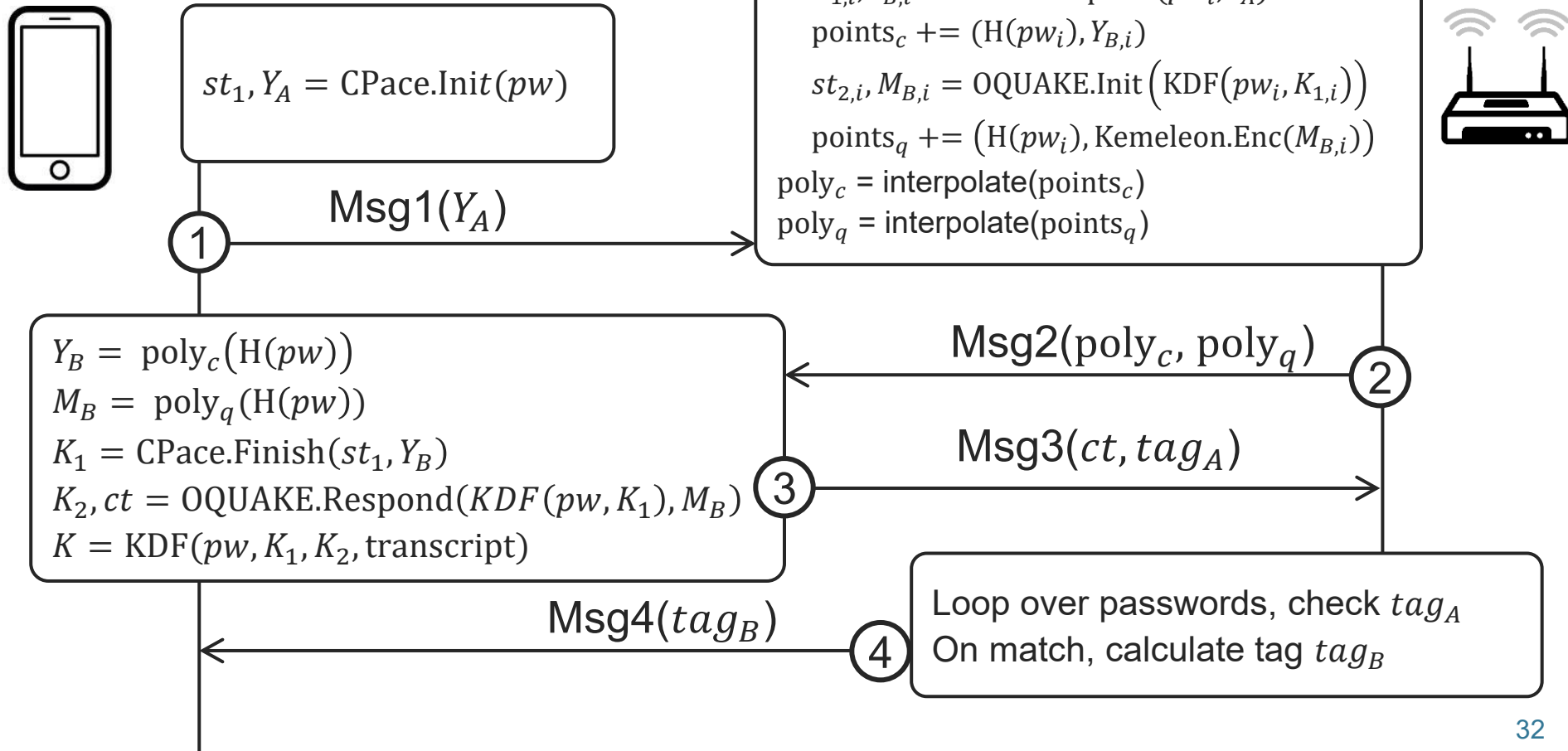
Msg2($\text{poly}_c, \text{poly}_q$)

3

Msg3(ct, tag_A)



Hybrid Multi-PW PAKE



Notes + security

- › This is currently still in the brainstorm phase – may have flaws
- › Can broadcast CPace polynomials to all clients at once
 - › Can even be sent outside the handshake...
 - › ...this makes supporting many passwords more feasible
 - › Note: on password change, *all* CPace messages must be regenerated

But is it secure? Hopefully.

- › **Looking for collaboration on design / security proof 😊**

Conclusion

Post-quantum PAKEs:

- › We need standardized ones! CPaceOQUAKE is promising.
- › Want to help IEEE 802.11? You can always message me.

Multi-password post-quantum PAKEs:

- › Hybrid construction looks feasible at first sight...
- › ...needs more research, **help on security proof welcome!**